# MLFlow

Suivi des Performances avec MLflow ..... 2
Prérequis : ..... 2
Introduction : ..... 2
Partie 1 : Qu'est-ce que MLflow ? ..... 2
Partie 2 : Structure du projet ..... 2
Partie 3 : Code minimal avec tracking ..... 3
Partie 4 : Lancer MLflow UI ..... 3
Partie 5 : Ajouter des paramètres ..... 4
Partie 6 : Automatiser le réentraînement avec Luigi (bonus) ..... 4
Enregistrer un modèle avec MLflow ..... 5
Charger un modèle avec MLflow ..... 5

# Suivi des Performances avec MLflow

## Prérequis :

- Environnement virtuel `(.venv)` configuré.
- Fichier requirements.txt contenant les dépendances nécessaires (mlflow, scikit-learn, pandas, matplotlib).
- Avoir un modèle de machine learning simple
- Fichier README.md décrivant l'installation et les routes de l'API.

## Introduction :

Ce projet vous guide dans l'utilisation de **MLflow**, un outil de tracking de machine learning, pour suivre les performances d'un modèle qui est régulièrement réentraîné avec de nouvelles données. L'objectif est de mesurer les changements de performance (ex: accuracy, RMSE) entre différentes versions du modèle.

### Partie 1 : Qu'est-ce que MLflow ?

MLflow est une plateforme open-source permettant de gérer le cycle de vie d'un projet de machine learning. Elle offre quatre composants principaux :

- **Tracking** : enregistrer les expériences, hyperparamètres, métriques et artefacts.
- **Projects** : organiser et reproduire des projets ML.
- **Models** : gérer les modèles pour les déployer facilement.
- **Registry** : versionner et approuver les modèles dans un registre central.

Dans ce projet, on se concentre sur **MLflow Tracking**.

### Partie 2 : Structure du projet

Créez un fichier `train.py` contenant le script de réentraînement et de tracking. L'idée est que chaque fois qu'on lance ce script, une nouvelle exécution est enregistrée.

### Partie 3 : Code minimal avec tracking

```
import mlflow
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error
from sklearn.model_selection import train_test_split
from sklearn.datasets import make_regression
# Générer un jeu de données de régression synthétique
X, y = make_regression(n_samples=1000, n_features=5,
noise=0.1, random_state=42)

X_train, X_test, y_train, y_test = train_test_split(X, y)

# MLflow tracking
with mlflow.start_run():
    model = LinearRegression()
    model.fit(X_train, y_train)
    preds = model.predict(X_test)

    mse = mean_squared_error(y_test, preds)
    mlflow.log_metric("mse", mse)
    mlflow.sklearn.log_model(model, "model")
```

### Partie 4 : Lancer MLflow UI

Dans le terminal:

```
mlflow ui
```

Allez sur http://localhost:5000 pour visualiser vos expériences : nom de run, timestamp, hyperparamètres, métriques, et version du modèle.

### Partie 5 : Ajouter des paramètres

On peut aussi sauvegarder les paramètres propres à l'expérience

```
with mlflow.start_run():
    alpha = 0.5  # exemple fictif si on utilisait Ridge
    mlflow.log_param("alpha", alpha)
```

### Partie 6 : Automatiser le réentraînement avec Luigi (bonus)

```
import luigi
import pandas as pd
import mlflow
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error
from sklearn.model_selection import train_test_split

class RetrainModel(luigi.Task):
    date = luigi.DateParameter()

    def output(self):
        return luigi.LocalTarget(f"models/model_{self.date}.txt")

    def run(self):
        # préparation des données
        # entrainement du modèle
        with mlflow.start_run():
            mlflow.log_metric("mse", mse)
            mlflow.sklearn.log_model(model, "model")

        with self.output().open("w") as f:
            f.write(f"Model trained on {self.date} with
mse={mse}\n")
```

Lancer le pipeline :

```
luigi --module mypipeline RetrainModel --date 2025-05-24 --local-
scheduler
```

## Enregistrer un modèle avec MLflow

```
import mlflow
import mlflow.sklearn
from sklearn.linear_model import LinearRegression
from sklearn.datasets import make_regression
from sklearn.model_selection import train_test_split

# Créer un jeu de données synthétique pour l'exemple
X, y = make_regression(n_samples=1000, n_features=5,
noise=0.1, random_state=42)
X_train, X_test, y_train, y_test = train_test_split(X, y,
test_size=0.2, random_state=42)

# Initialiser et entraîner le modèle
model = LinearRegression()
model.fit(X_train, y_train)

# Enregistrer le modèle dans MLflow
# Démarre une nouvelle exécution MLflow
with mlflow.start_run():
    # Enregistrer le modèle
    mlflow.sklearn.log_model(model, "linear_regression_model")
   # Optionnel : enregistrer des paramètres
    mlflow.log_param("model_type", "linear_regression")
    print("Le modèle a été enregistré avec succès dans
MLflow.")
```

## Charger un modèle avec MLflow

```
# Charger le modèle depuis MLflow
loaded_model =
mlflow.sklearn.load_model("runs:/<run_id>/linear_regression_mo
del")
# Faire des prédictions avec le modèle chargé
predictions = loaded_model.predict(X_test)
print("Predictions : ", predictions)
```